

Roadmap de Desarrollo & Plan de Implementación: iriome.build (2026)

Este documento guarda el diseño y la evolución de los proyectos de Iriome Naranjo, evaluados bajo una mirada de Reclutador IT Senior.

FASE 1: EVALUACIÓN INICIAL DEL ROADMAP (PDF)

1. Análisis del Roadmap Completo

El documento *"DESAFÍOS & ROADMAP DE DESARROLLO"* presenta una progresión lógica que va desde consolidar los fundamentos de bases de datos/lógica (Habitly, Calculadora de Propinas) hacia el aprendizaje de nuevas herramientas (Kotlin), automatizaciones avanzadas de procesos y diseño de sistemas distribuidos o de alta concurrencia.

FASE 2: DEFINICIÓN DE PROYECTO - RETO 01 (KOTLIN)

Estado: Definición completada (Foco actual: Diseñar un proyecto backend robusto usando Kotlin que sirva como evidencia de dominio técnico).

Historial de Preguntas y Respuestas (Bitácora de Diseño)

- **Pregunta 1:** ¿Qué tipo de aplicación backend prefieres desarrollar para este reto?
- **Respuesta de Iriome:** «Con Kotlin aún estoy asentando bases, aprendiendo el programa. Pero lo primero que podría hacer es un catálogo por ejemplo, como una PokeApi, que es un proyecto común, pero para cambiar y destacar, lo haré de otra cosa (necesito que propongas ideas).»
- **Pregunta 2:** ¿Con qué framework del ecosistema de Kotlin te gustaría trabajar?
- **Respuesta de Iriome:** «Pues me gustaría trabajar con Spring Boot, así lo repaso un poco.»
- **Pregunta 3:** ¿Te interesa que el proyecto implemente corrutinas (Coroutines) y flujos asíncronos (Flow) de Kotlin desde el primer momento?
- **Respuesta de Iriome:** «Cómo no sé que es esto, lo dejaré para un futuro en proyectos más avanzados.»
- **Pregunta 4:** ¿Cuál de estas ideas te convence más para desarrollar?
- **Respuesta de Iriome:** «La segunda idea me gusta mucho. Además fortalece mi origen canario y me siento orgulloso de ello. Canary Wine & Guachinches API (Bodegas y Gastronomía)»
- **Pregunta 5:** ¿Qué base de datos prefieres usar para la persistencia del catálogo?
- **Respuesta de Iriome:** «Quiero usar PostgreSQL mediante pgAdmin 4»
- **Pregunta 6:** ¿Qué entidades o tablas principales quieres que maneje tu base de datos?
- **Respuesta de Iriome:** «Vale, empecemos con esa recomendación [Guachinche, Vino, Plato] (recuerda que este proyecto estará en el futuro).»
- **Pregunta 7:** ¿Añadimos alguna regla de negocio realista para impresionar a los evaluadores?
- **Respuesta de Iriome:** «Venga vale, todo lo que sea impresionar y mejorar adelante.»
- **Pregunta 8:** ¿Cómo planeas presentar y documentar la API para que un reclutador pueda probarla sin esfuerzo?
- **Respuesta de Iriome:** «Para la opción A [Swagger] ¿yo tendría que tener la app funcionando desde mi ordenador o puede ser extraíble desde el repositorio de github que se hará?»
- **Pregunta 9 (Despliegue e Interactividad):** ¿Prefieres despliegue online, vista visual en el portfolio, pruebas JUnit y flujo de Git privado?
- **Respuesta de Iriome:** «Hagamos la opción del Online. Vale, haremos eso, una pequeña vista visual en el proyecto. Sí las pruebas JUnit. Al principio será privado, pero aún así se harán los commits ahí para que se pueda ver el proceso.»

Blueprint Arquitectónico: Canary Wine & Guachinches API

1. Resumen y Propósito

Desarrollar una API REST premium utilizando **Kotlin + Spring Boot** conectada a una base de datos relacional **PostgreSQL**, encargada de catalogar guachinches tradicionales de las Islas Canarias, vinos con denominación de origen y platos de la gastronomía típica. El valor diferencial radica en un motor de validación de reglas de negocio que modela de forma estricta las normativas legales de apertura y oferta de estos establecimientos (cumplimiento regulatorio backend).

2. Stack Tecnológico

- **Lenguaje:** Kotlin 2.x (aprovechando null-safety, data classes para DTOs, e inmutabilidad con variables read-only `val`).
- **Framework:** Spring Boot (Spring Web MVC, Spring Data JPA).
- **Base de Datos:** PostgreSQL (con pgAdmin 4 para diseño y scripting SQL).
- **Documentación:** Springdoc OpenAPI (Swagger UI integrado).
- **Testing:** JUnit 5 + MockMvc (Spring Boot Test).
- **Despliegue (DevOps):** Render / Railway / [Fly.io](https://fly.io) (Hosting gratuito conectado a GitHub con base de datos administrada Postgres).

3. Estructura de Datos (Modelo de Base de Datos)

- **Guachinche** : ID, nombre, municipio, direccion, fechaApertura, mesesAbierto (int), cosecheroProductor (boolean).
- **Vino** : ID, nombre, tipo (Enum: TINTO, BLANCO, ROSADO), variedadUva, denominacionOrigen (Enum: TACORONTE, VALLE_GUIMAR, LANZAROTE, etc.), precioBotella (BigDecimal).
- **Plato** : ID, nombre, categoria (Enum: ENTRANTE, PRINCIPAL, POSTRE), ingredientesPrincipales.
- **Relaciones (JPA):**
 - Un **Guachinche** sirve una lista de **Plato** (Relación Many-to-Many o One-to-Many).
 - Un **Guachinche** vende una lista de **Vino** (Relación Many-to-Many o One-to-Many).

4. Motor de Validación: Reglamento Canario de Guachinches (Reglas de Negocio)

Para demostrar capacidad de desarrollo backend robusta y orientada a dominios de negocio, el servicio implementará las siguientes reglas estrictas:

- **Límite de Tiempo de Apertura:** Un establecimiento de tipo **Guachinche** no puede estar abierto más de **4 meses** consecutivos al año. Al intentar registrar una apertura con `mesesAbierto > 4` , la API rechazará la petición con un error `400 Bad Request` y un mensaje descriptivo.
- **Límite de Bodega (Vino propio):** Un guachinche solo puede ofrecer un **máximo de 3 vinos diferentes** (cosecha propia). La API controlará las inserciones en la relación Guachinche-Vino.
- **Límite de Carta (Platos cocinados):** Un guachinche solo puede ofrecer un **máximo de 3 platos elaborados** simultáneamente en su carta. Intentar añadir un 4º plato disparará una excepción personalizada `ReglamentoGuachincheException` que se manejará globalmente con un `@ControllerAdvice` para responder un JSON estructurado con el código de error correspondiente.

5. Plan de Demostración & Frontend en el Portfolio

- **Swagger UI Online:** La API se desplegará en la nube (ej. Render) de modo que al pulsar en un enlace en el portfolio de Iriome, se abra la web interactiva de Swagger para probar endpoints en vivo.
- **Visualizador en Portfolio Web:** Se integrará una sección interactiva en tu portfolio (con AJAX/Fetch en [script.js](file:///c:/Users/iriom/OneDrive/Documentos/GitHub/Portfolio/src/js/script.js)) que consultará directamente a la API desplegada para mostrar un listado/galería estética de los guachinches registrados de forma visual, cerrando la brecha para los reclutadores no técnicos.
- **Semillero de Datos (Seeding):** Un archivo `resources/data.sql` con guachinches y platos reales rellenará automáticamente la base de datos local y de producción al iniciar por primera vez.

6. Aseguramiento de Calidad (Testing)

Se integrará una suite de pruebas unitarias y de integración usando **JUnit 5** y **MockMvc** para verificar de forma automática:

- Los flujos normales de obtención y registro de datos (HTTP 200/201).
- El bloqueo correcto de violaciones del reglamento (HTTP 400 con JSON de error controlado al intentar añadir un 4º plato o 4º vino).

7. Flujo de Git & Repositorio

- El repositorio nacerá de forma privada en GitHub para desarrollar con tranquilidad, pero se gestionará bajo la metodología de **commits atómicos y descriptivos en español**.
- Al finalizar el proyecto, el repositorio se cambiará a público para que los reclutadores y líderes técnicos puedan auditar la calidad del código Kotlin desarrollado.